

R3 (Reactive Extensions for C#)

R&D 기술 문서 v2

바삭한소프트 기업협약 프로젝트

출처: <https://github.com/Cysharp/R3>

1. Reactive 란 무엇인가

Reactive 는 '반응형'이라는 뜻입니다.

□ 비유: 엑셀 스프레드시트를 떠올려보세요. A1 셀 숫자를 바꾸면 A1 을 참조하는 B1, C1 이 자동으로 바뀝니다. 내가 일일이 'B1 도 바뀌, C1 도 바뀌'라고 명령하지 않아도요. Reactive Programming 이 딱 이겁니다.

게임으로 바꾸면 — 직원 레벨이 올라가면, 레벨을 표시하는 UI 텍스트가 자동으로 바뀌는 것. 레벨 올리는 코드 따로, UI 업데이트 코드 따로 작성하지 않아도 되는 것. 이게 Reactive 의 핵심입니다.

2. R3 란 무엇인가

R3 는 C#에서 Reactive Programming 을 쉽게 쓸 수 있게 만든 라이브러리입니다.

Unity 전용으로 만들어진 기존 UniRx 라이브러리를 개선한 후속 버전으로, 더 빠르고 현대적인 C# 문법을 지원합니다.

구분	설명
UniRx	Unity 전용 Reactive 라이브러리. 오래된 설계 방식 사용.

구분	설명
R3	UniRx 의 후속작. 더 빠른 성능, 현대적 설계. Unity 포함 여러 플랫폼 지원.

i 한 줄 정리: R3 = Unity 에서 쓸 수 있는 '값이 바뀌면 자동으로 반응하는' 기능 모음

3. Observable 과 Observer — 기존 이벤트와 뭐가 다른가

3.1 C# 기존 델리게이트/이벤트와의 차이

둘 다 '어떤 일이 일어나면 알려줘' 라는 목적은 같습니다. 차이는 가공 능력에 있습니다.

방식	특징
델리게이트 / 이벤트	버튼 누르면 함수 실행. 단순 호출. 중간 가공 불가.
Observable / Observer	값이 흘러가는 파이프라인. 중간에 필터, 변환, 지연 등 가공 가능.

□ 비유: 수도관을 생각해 보세요. 델리게이트는 밸브 하나. Observable 은 중간에 필터, 정수기, 유량 조절기 등을 붙일 수 있는 파이프라인입니다.

3.2 구독(Subscribe)과 구독 해지(Dispose)

Observable 은 파이프라인이고, Subscribe 는 '나 이 파이프 볼게요' 라고 등록하는 행위입니다.

Dispose 는 '나 이제 안 봐도 돼요' 라고 등록을 해제하는 것입니다.

△ 구독 해지를 하지 않으면, 오브젝트가 화면에서 사라져도 구독이 살아남아 계속 메모리를 잡아먹습니다. 반드시 OnDestroy 에서 Dispose 를 호출해야 합니다.

4. ReactiveProperty — 핵심 기능

4.1 ReactiveProperty 란

ReactiveProperty 는 '이벤트 시스템이 내장된 변수 공간'입니다.

일반 변수는 값만 저장합니다. ReactiveProperty 는 값 저장 + 값이 바뀔 때 구독자에게 자동 알림 기능까지 포함되어 있습니다.

```
// 일반 변수: 값만 저장. 바뀌어도 아무도 모름
int Level = 1;

// ReactiveProperty: 값 저장 + 바뀌면 자동으로 알림
ReactiveProperty<int> Level = new ReactiveProperty<int>(1);
```

선언 방식을 풀어보면:

- ReactiveProperty — '알림 기능이 내장된 변수 공간'을 할당하겠다는 선언
- <int> — 이 공간에 담을 값의 타입
- (1) — 초기값

4.2 기본 사용 흐름

```
// 1. 선언
var Level = new ReactiveProperty<int>(1);

// 2. 구독: '레벨이 바뀌면 이 코드 실행해줘' 등록
Level.Subscribe(lv => levelText.text = $"Lv.{lv}");

// 3. 값 변경: 자동으로 UI 가 바뀜
Level.Value = 2; // → levelText 가 자동으로 "Lv.2" 로 변경됨
```

i 같은 값을 다시 넣으면 알림이 발생하지 않습니다. 예: Level.Value = 2 를 두 번 해도 알림은 한 번만 옵니다. 강제로 알림을 보내고 싶다면 Level.OnNext(2)를 사용합니다.

4.3 게임 회사 키우기 적용 예시

직원 레벨이 오르면 UI 가 자동으로 업데이트되는 구조입니다.

```
// 직원 데이터
var employeeLevel = new ReactiveProperty<int>(1);

// UI 연결: 레벨 바뀌면 자동 반영
employeeLevel.Subscribe(lv => levelText.text = $"Lv.{lv}");
```

```
// 레벨업 버튼 클릭 시
employeeLevel.Value += 1; // UI 가 자동으로 바뀜
```

5. Subject — 이벤트 발행

5.1 Subject 란

Subject 는 '내가 직접 값을 쓸 수 있는 Observable'입니다.

ReactiveProperty 가 값을 보관하는 그릇이라면, Subject 는 값을 보관하지 않고 그냥 이벤트를 쏘는 발사대입니다.

```
var onHire = new Subject<string>(); // 직원 고용 이벤트 발사대

// 구독: '고용 이벤트 오면 이 코드 실행해줘'
onHire.Subscribe(name => Debug.Log($"{name} 고용 완료!"));

// 이벤트 발행
onHire.OnNext("김개발"); // → '김개발 고용 완료!' 출력
```

i Subject 사용 방법이 반드시 저 형태일 필요는 없습니다. OnNext()로 값 발행 → Subscribe()로 받는다, 이 두 가지가 핵심이고 나머지는 상황에 따라 달라집니다.

5.2 Subject 종류

종류	특징
Subject<T>	기본형. 값을 보관하지 않음. OnNext 시점에 구독 중인 Observer 에게만 전달.
ReactiveProperty<T>	값을 보관함. 새 구독자도 즉시 현재 값을 받음. 중복 값은 알림 없음. UI 바인딩에 주로 사용.

6. 구독 관리 — 메모리 누수 방지

구독(Subscribe)을 등록하면 반드시 나중에 해제(Dispose)해야 합니다. 해제하지 않으면 오브젝트가 파괴되어도 구독이 살아남아 메모리 낭비가 발생합니다.

□ 비유: 넷플릭스 구독을 해지하지 않으면 계속 요금이 나가는 것처럼, Subscribe 를 Dispose 하지 않으면 계속 메모리를 잡아먹습니다.

6.1 기본 패턴 — MonoBehaviour 에서의 처리

```
public class GameUI : MonoBehaviour
{
    IDisposable _disposable;

    void Start()
    {
        // 구독 등록
        _disposable = employeeLevel.Subscribe(lv => levelText.text = $"Lv.{lv}");
    }

    void OnDestroy()
    {
        _disposable.Dispose(); // 오브젝트 파괴 시 구독 해제
    }
}
```

6.2 구독이 여러 개일 때 — Disposable.Combine

구독이 여러 개라면 Disposable.Combine 으로 묶어서 한 번에 해제할 수 있습니다.

```
void Start()
{
    var d1 = employeeLevel.Subscribe(lv => levelText.text = $"Lv.{lv}");
    var d2 = employeeName.Subscribe(name => nameText.text = name);
    var d3 = money.Subscribe(m => moneyText.text = $"{m}원");

    _disposable = Disposable.Combine(d1, d2, d3); // 하나로 묶기
}

void OnDestroy()
{
    _disposable.Dispose(); // 한 번에 전부 해제
}
```

7. Unity 설치 방법

Package Manager 에서 Git URL 로 설치합니다.

```
// Unity Package Manager → Add package from git URL 에 아래 URL 입력  
https://github.com/Cysharp/R3.git?path=src/R3.Unity/Assets/R3.Unity
```

△ 미션 가이드 기준 Unity 버전: 6000.0.65f1 — 팀 전체가 동일한 버전을 사용해야 합니다.

8. 사용 전 체크리스트

항목	내용
✓using R3 선언	각 파일 상단에 using R3; 추가 필요
✓구독 해제	OnDestroy 에서 반드시 Dispose() 호출
✓Unity 버전 통일	팀 전체 동일 Unity 버전(6000.0.65f1) 사용 확인
✓같은 값 알림	같은 값도 알림이 필요하면 .Value 대신 .OnNext() 사용

9. 참고 자료

- R3 공식 GitHub: <https://github.com/Cysharp/R3>
- ReactiveX 개념 설명: <https://reactivex.io/intro.html>
- Introduction to Rx.NET (영문): <https://introtorx.com/>
- R3 설계 철학 블로그 (영문): <https://neuecc.medium.com/r3-a-new-modern-reimplementation-of-reactive-extensions-for-c-cf29abcc5826>

※ 본 문서는 R3 공식 GitHub README 및 공식 문서를 기반으로 작성되었습니다.